

```

# -*- coding: utf-8 -*-
"""Joins_And_window_functions.ipynb

Automatically generated by Colab.

Original file is located at
https://colab.research.google.com/drive/1tA\_e0TSrCUa-F4EVxPu2ITCLY9p51ifg
"""

import pandas as pd
import sqlite3

# Initialize in-memory SQLite database
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# 1. Employees Table (For Self Join)
employees_data = {
    "Emp_ID": [1, 2, 3, 4],
    "Emp_Name": ["Alice (CEO)", "Bob (Manager)", "Charlie (Dev)", "David (Dev)"],
    "Manager_ID": [None, 1, 2, 2] # Alice manages Bob; Bob manages Charlie & David
}

# 2. Customers Table
customers_data = {
    "Customer_ID": [101, 102, 103, 104],
    "Customer_Name": ["Rajat", "Sita", "John", "Amit"]
}

# 3. Orders Table
orders_data = {
    "Order_ID": [5001, 5002, 5003, 5004],
    "Customer_ID": [101, 102, 101, 999], # 999 is an orphaned record (no customer)
    "Amount": [250.00, 45.50, 120.00, 89.99]
}

# 4. Form Factor Matrix (For Cross Join)
products = {"Product_Name": ["Laptop", "Phone"]}
colors = {"Color": ["Black", "Silver"]}

# Load dataframes to SQL
pd.DataFrame(employees_data).to_sql('Employees', conn, index=False, if_exists='replace')
pd.DataFrame(customers_data).to_sql('Customers', conn, index=False, if_exists='replace')
pd.DataFrame(orders_data).to_sql('Orders', conn, index=False, if_exists='replace')
pd.DataFrame(products).to_sql('Products', conn, index=False, if_exists='replace')
pd.DataFrame(colors).to_sql('Colors', conn, index=False, if_exists='replace')

print("Database initialized with sample tables: Employees, Customers, Orders, Products, Colors.")

print("--- 1. INNER JOIN (Only matching records) ---")
# Drops Customer 103, 104 (no orders) and Order 5004 (no matching customer)
query_inner = """
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Amount
FROM Customers c
INNER JOIN Orders o ON c.Customer_ID = o.Customer_ID;
"""
display(pd.read_sql_query(query_inner, conn))

print("\n--- 2. LEFT JOIN (Keeps all customers) ---")
# Retains John and Amit even though they spent $0
query_left = """
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Amount
FROM Customers c
LEFT JOIN Orders o ON c.Customer_ID = o.Customer_ID;
"""
display(pd.read_sql_query(query_left, conn))

print("\n--- 3. RIGHT JOIN emulation (Keeps all orders) ---")
# Retains Order 5004 even though Customer_ID 999 is missing from the customer registry
query_right = """
SELECT o.Order_ID, o.Amount, c.Customer_ID, c.Customer_Name
FROM Orders o
LEFT JOIN Customers c ON o.Customer_ID = c.Customer_ID;
"""
display(pd.read_sql_query(query_right, conn))

```

```

print("--- 4. SELF JOIN (Hierarchical Reporting Structure) ---")
query_self = """
SELECT
    e.Emp_Name AS Employee,
    m.Emp_Name AS Reports_To
FROM Employees e
LEFT JOIN Employees m ON e.Manager_ID = m.Emp_ID;
"""
display(pd.read_sql_query(query_self, conn))

print("\n--- 5. CROSS JOIN (Cartesian Product Matrix) ---")
query_cross = """
SELECT p.Product_Name, c.Color
FROM Products p
CROSS JOIN Colors c;
"""
display(pd.read_sql_query(query_cross, conn))

# Emulating physical storage pages from data blocks
build_table_customers = [
    {"id": 101, "name": "Rajat"},
    {"id": 102, "name": "Sita"},
    {"id": 103, "name": "John"}
]

probe_table_orders = [
    {"order_id": 5001, "cust_id": 101, "amt": 250.0},
    {"order_id": 5002, "cust_id": 102, "amt": 45.5},
    {"order_id": 5004, "cust_id": 999, "amt": 89.9} # Will hash-miss
]

def physical_hash_join(build_source, probe_source, build_key, probe_key):
    print("=== STARTING PHYSICAL HASH JOIN EXECUTION ===")

    # PHASE 1: Build Phase
    # Creating an in-memory hash index map on the join key
    hash_table = {}
    print(f"\n[Phase 1: Build] Hashing Build Source on Key: '{build_key}'")
    for row in build_source:
        key = row[build_key]
        if key not in hash_table:
            hash_table[key] = []
        hash_table[key].append(row)
        print(f" -> Hashed Key {key} mapping to Row: {row}")

    # PHASE 2: Probe Phase
    # Streaming through the larger table and checking against the hash bucket index
    joined_results = []
    print(f"\n[Phase 2: Probe] Streaming Probe Source on Key: '{probe_key}'")
    for probe_row in probe_source:
        match_key = probe_row[probe_key]
        print(f" Probing Key: {match_key}...", end="")

        if match_key in hash_table:
            print(" [CACHE HIT] Matching records found!")
            for match_row in hash_table[match_key]:
                # Combine memory fields
                combined_row = (**match_row, **probe_row)
                joined_results.append(combined_row)
        else:
            print(" [CACHE MISS] No corresponding key in Hash Index. Skipping record.")

    print("\n=== EXECUTION COMPLETED ===")
    return pd.DataFrame(joined_results)

# Run the physical optimization trace engine
df_hash_join_output = physical_hash_join(
    build_source=build_table_customers,
    probe_source=probe_table_orders,
    build_key="id",
    probe_key="cust_id"
)

print("\nFinal Result Matrix generated by Physical Hash Join:")
df_hash_join_output

```

Database initialized with sample tables: Employees, Customers, Orders, Products, Colors.

--- 1. INNER JOIN (Only matching records) ---

	Customer_ID	Customer_Name	Order_ID	Amount
0	101	Rajat	5001	250.0
1	101	Rajat	5003	120.0
2	102	Sita	5002	45.5

--- 2. LEFT JOIN (Keeps all customers) ---

	Customer_ID	Customer_Name	Order_ID	Amount
0	101	Rajat	5001.0	250.0
1	101	Rajat	5003.0	120.0
2	102	Sita	5002.0	45.5
3	103	John	NaN	NaN
4	104	Amit	NaN	NaN

--- 3. RIGHT JOIN emulation (Keeps all orders) ---

	Order_ID	Amount	Customer_ID	Customer_Name
0	5001	250.00	101.0	Rajat
1	5002	45.50	102.0	Sita
2	5003	120.00	101.0	Rajat
3	5004	89.99	NaN	None

--- 4. SELF JOIN (Hierarchical Reporting Structure) ---

	Employee	Reports_To
0	Alice (CEO)	None
1	Bob (Manager)	Alice (CEO)
2	Charlie (Dev)	Bob (Manager)
3	David (Dev)	Bob (Manager)

--- 5. CROSS JOIN (Cartesian Product Matrix) ---

	Product_Name	Color
0	Laptop	Black
1	Laptop	Silver
2	Phone	Black
3	Phone	Silver

=== STARTING PHYSICAL HASH JOIN EXECUTION ===

[Phase 1: Build] Hashing Build Source on Key: 'id'

-> Hashed Key 101 mapping to Row: {'id': 101, 'name': 'Rajat'}
-> Hashed Key 102 mapping to Row: {'id': 102, 'name': 'Sita'}
-> Hashed Key 103 mapping to Row: {'id': 103, 'name': 'John'}

[Phase 2: Probe] Streaming Probe Source on Key: 'cust_id'

Probing Key: 101... [CACHE HIT] Matching records found!
Probing Key: 102... [CACHE HIT] Matching records found!
Probing Key: 999... [CACHE MISS] No corresponding key in Hash Index. Skipping record.

=== EXECUTION COMPLETED ===

Final Result Matrix generated by Physical Hash Join:

	id	name	order_id	cust_id	amt
0	101	Rajat	5001	101	250.0
1	102	Sita	5002	102	45.5

```
import pandas as pd
# EXECUTE SETUP TO INSTITUTE DATA CONTEXTS
servers_inventory = {
    "Host_ID": ["SRV-01", "SRV-02", "SRV-03"],
    "Role": ["Web Front", "API Gateway", "Database Replica"]
}
```

```

live_interfaces = {
    "Interface_ID": ["eth0", "eth1"],
    "Mapped_Host": ["SRV-01", "SRV-02"],
    "IP_Address": ["10.0.0.4", "10.0.0.9"]
}

df_srv = pd.DataFrame(servers_inventory)
df_inf = pd.DataFrame(live_interfaces)

df_srv.to_sql('Servers', conn, index=False, if_exists='replace')
df_inf.to_sql('Interfaces', conn, index=False, if_exists='replace')

# Broken Query Framework
faulty_query = """
SELECT s.Host_ID, s.Role, i.Interface_ID, i.IP_Address
FROM Servers s
INNER JOIN Interfaces i ON s.Host_ID = i.Mapped_Host;
"""

print("--- Faulty Output Metrics (Missing Database Replica SRV-03) ---")
pd.read_sql_query(faulty_query, conn)

correct_query = """
SELECT s.Host_ID, s.Role, i.Interface_ID, i.IP_Address
FROM Servers s
LEFT JOIN Interfaces i ON s.Host_ID = i.Mapped_Host;
"""

pd.read_sql_query(correct_query, conn)

"""1. Explain why SRV-03 is excluded from the query framework above based on the mechanics of an INNER JOIN.
-- INNER Join works on a principle of joining only the matching rows in both tables. Since SRV-03 row is not available in

2. Rewrite the query below to implement a join strategy that outputs all server configurations from inventory, regardless
"""

--- Faulty Output Metrics (Missing Database Replica SRV-03) ---
'1. Explain why SRV-03 is excluded from the query framework above based on the mechanics of an INNER JOIN.\n -- INNER Join
works on a principle of joining only the matching rows in both tables. Since SRV-03 row is not available in live_interface
table, record is not fetched in result. \n\n2. Rewrite the query below to implement a join strategy that outputs all serve
r configurations from inventory regardless of whether they have a live network interface \n'

```